Comandare una radio Yaesu FT-847 con un CYD (ESP32 "Cheap Yellow Display")

Risposta breve: Sì, è assolutamente possibile. Il CYD è una scheda basata su ESP32-WROOM-32 con display TFT touch da 2,8", e l'FT-847 dispone di una porta **CAT (Computer Aided Transceiver)** in RS-232C su connettore **DB-9**. Basta convertire i livelli logici (3,3 V TTL dell'ESP32 ↔ RS-232C dell'FT-847) con un piccolo adattatore **MAX3232** e scrivere un firmware che parla il protocollo CAT dell'FT-847. Con questo si comandano frequenza, modo, PTT, CTCSS/DCS, shift/offset di repeater e si leggono S-meter, potenza e stato TX.

Quella che segue è una guida tecnica completa: caratteristiche della porta CAT, protocollo, hardware necessario, schema di collegamento, scelta dei pin liberi del CYD, architettura software e uno sketch Arduino di esempio pronto da adattare.

---

1. Perché è fattibile (in sintesi)

Requisito	FT-847	CYD (ESP32-2432S028R)	Esito
Interfaccia di controllo	CAT su DB-9, RS-232C	3 UART interne (UART0/1/2)	✅ serve adattatore di livello
Livelli logici	RS-232C (±5…±12 V)	3,3 V TTL	⚠ serve MAX3232
Velocità	4800 / 9600 / 57600 bps, 8N2	qualsiasi, impostabile	✅
Protocollo	5 byte, opcode finale	software custom	✅
Interfaccia utente	—	TFT 240×320 + touch	✅
Alimentazione	13,8 V (radio)	5 V USB	✅ indipendenti

Il punto delicato **non** è la potenza di calcolo (l'ESP32 è enormemente sovradimensionato per inviare 5 byte ogni tanto), ma **l'adattamento dei livelli elettrici** e la **corretta piedinatura** del cavo.

---

2. La porta CAT dell'FT-847

Dal manuale operativo Yaesu (connettore n. 4 del pannello posteriore):

> *"This DB-9 connector is used for interconnection to a personal computer's serial port to enable computer-assisted operation. Use a 'null modem' (not 'straight') RS-232C serial data cable; no 'level converter' or other hardware interface should be required."*

E nella sezione *CAT System Programming*:

> *"The FT-847 has a built-in level converter, allowing direct connection from the rear panel CAT jack to the serial port of your computer, without the need for an external RS-232C level converter box."*

Punti chiave:

- **Connettore:** DB-9 femmina sul pannello posteriore.
- **Livelli:** RS-232C veri (l'FT-847 ha già il suo convertitore di livello interno). Non sono livelli TTL: **non collegare mai direttamente l'ESP32 al DB-9**, si rischia di danneggiare il GPIO.
- **Cavo:** *null-modem* (incrociato), cioè i pin **2 e 3 vanno scambiati** (TX↔RX) e il pin **5 = GND** diretto. È confermato da Yaesu e da decine di utenti (serve un adattatore/cavo crossover, non dritto).
- **Formato seriale:** 1 bit di start, **8 bit di dati, nessuna parità, 2 bit di stop** (in breve **8N2**).
- **Velocità:** impostata dal **Menu #37 [CAT RATE]** con valori **4800 / 9600 / 57600 bps** (default **4800**). Deve combaciare con quella dell'ESP32.
- **Nota storica:** le prime FT-847 (numeri di serie bassi) **non** avevano la capacità di *lettura* dello stato (S-meter, ecc.); su queste la lettura non risponde. La scrittura (frequenza, modo, PTT) funziona comunque.

2.1 Protocollo CAT: blocchi da 5 byte

Il protocollo dell'FT-847 è quello "vecchio" Yaesu a **5 byte**, con **opcode nell'ultimo byte** (attenzione: è diverso dalle FT-817/857/897 dove l'opcode è il *primo* byte).

```
Byte:     1        2        3        4        5
        [dato]   [dato]   [dato]   [dato]   [OPCODE]
        MSD <———————— BCD ————————> LSD
```

- I **primi 4 byte** sono argomenti (dati BCD o valori "dummy" di riempimento).
- Il **5° byte** è l'opcode dell'istruzione.
- Tra un byte e il successivo sono ammessi fino a **200 ms**; si possono quindi inviare i 5 byte di fila.
- Tutti i valori sono **esadecimali**.

Codifica della frequenza: i primi 4 byte contengono la frequenza espressa in **BCD, big-endian** (byte più significativo per primo) e in unità di **10 Hz**, cioè `frequenza_Hz / 10`. Esempio dal manuale: per **432,1000 MHz** i quattro byte sono `43 21 00 00` (→ `43210000` × 10 Hz = 432,1 MHz), con opcode `01`.

Esempio: **28,150 MHz** → `28.150.000 / 10 = 2.815.000` → `02 81 50 00`, opcode `01`. Sequenza completa: `02 81 50 00 01`.

2.2 Tabella dei comandi (opcode = 5° byte)

Comando	Byte 1	Byte 2-4	Opcode (5° byte)
CAT ON / OFF	—	dummy	`00` / `80`
PTT ON (TX) / OFF (RX)	—	dummy	`08` / `88`
Satellite ON / OFF	—	dummy	`4E` / `8E`
Set Frequency	BCD	BCD	`01` (MAIN), `11` (SAT RX), `21` (SAT TX)
Operating Mode	D1	dummy	`07` (MAIN), `17` (SAT RX), `27` (SAT TX)
CTCSS/DCS Mode	D1	dummy	`0A` / `1A` / `2A`
CTCSS Frequency	D1	dummy	`0B` / `1B` / `2B`
DCS Code	BCD	dummy	`0C` / `1C` / `2C`
Repeater Shift	—	dummy	`09` (−), `49` (+), `89` (simplex)
Repeater Offset	BCD	BCD	`F9`
Read RX status	—	—	`E7` (1 byte di risposta)
Read TX status	—	—	`F7` (1 byte di risposta)
Read Freq+Mode	—	—	`03` / `13` / `23` (5 byte di risposta)

Codici modo (D1): `00`=LSB, `01`=USB, `02`=CW, `03`=CW-R, `04`=AM, `08`=FM, `82`=CW-N, `83`=CW-R-N, `84`=AM-N, `88`=FM-N.

Byte di risposta:

- `E7` (RX status, 1 byte): bit 0-4 = S-meter (0…31), bit 5 = discriminatore/modo, bit 6 = stato CTCSS, bit 7 = squelch (chiuso/aperto).
- `F7` (TX status, 1 byte): bit 0-4 = potenza TX, bit 5 = split on/off, bit 6 = SWR elevato, bit 7 = PTT (0 = TX, 1 = RX).
- `03` / `13` / `23` (freq+mode, 5 byte): byte 1-4 = frequenza BCD (×10 Hz), byte 5 = codice modo.

---

3. Cosa serve (lista materiali)

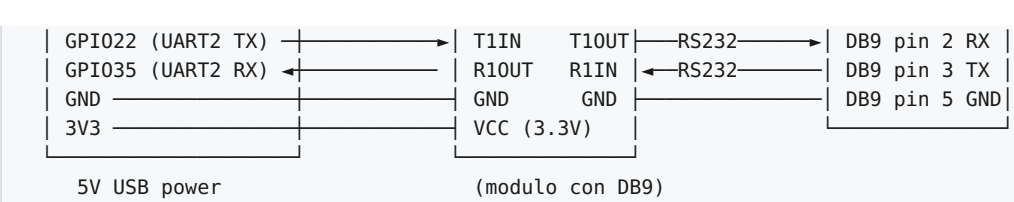
1. **CYD** — scheda **ESP32-2432S028R** (2,8" TFT + touch).
2. **Modulo convertitore RS-232 ↔ TTL basato su MAX3232** (versione **3,3 V**). *Non* usare un MAX232 (5 V): il MAX3232 tollera la logica a 3,3 V ed è quello giusto per l'ESP32. In alternativa un MAX3221/SP3232.
3. **Cavo/adattatore DB-9 null-modem** (crossover) oppure un connettore DB-9 femmina + cavo per autocostruire il collegamento.
4. **Alimentazione 5 V USB** per il CYD (e 13,8 V per la radio, ovviamente).
5. (Opzionale) un piccolo box e qualche filo dupont.

> ⚠ **Non** collegare TX/RX dell'ESP32 direttamente al DB-9 dell'FT-847. Serve sempre il MAX3232 in mezzo.

---

4. Schema di collegamento

```
  CYD (ESP32-2432S028R)          MAX3232 (3.3V)          Yaesu FT-847
 ┌──────────────────────┐      ┌──────────────┐      ┌──────────────────┐
```



Note pratiche:

- Il MAX3232 trasforma la logica **3,3 V TTL** dell'ESP32 in **RS-232C** (e viceversa), che è ciò che l'FT-847 si aspetta.
- Poiché l'FT-847 richiede un cavo **null-modem** (crossover) verso un PC, e i moduli MAX3232 hanno in genere il DB-9 cablato "tipo PC", **interponi un adattatore null-modem** tra modulo e radio (oppure incrocia tu TX/RX).
- Se la comunicazione non parte, **scambia tra loro i due fili di dati** (TX↔RX): è l'errore più comune. Non si danneggia nulla.
- **Massa comune obbligatoria** tra CYD, MAX3232 e FT-847.

(Vedi l'immagine `schema_collegamento.png` allegata.)

---

5. Quali pin usare sul CYD

Il CYD è "povero" di GPIO liberi perché display, touch e SD occupano gran parte dei pin. Situazione tipica (ESP32-2432S028R):

- **Display (SPI):** MISO 12, MOSI 13, SCLK 14, CS 15, DC 2, BL 21
- **Touch (SPI):** IRQ 36, MOSI 32, MISO 39, CLK 25, CS 33
- **microSD (SPI):** MISO 19, MOSI 23, SCK 18, CS 5
- **LED RGB:** R 4, G 16, B 17 · **LDR:** 34 · **Speaker:** 26

Pin realmente disponibili sui connettori P3/CN1:

Connettore	Pin esposti
P3	GND, **GPIO35**, **GPIO22**, GPIO21 (retroilluminazione)
CN1	GND, **GPIO22**, **GPIO27**, 3V3

Scelta consigliata per la UART2 (CAT):

- **TX = GPIO22** (uscita, libero e ben esposto su P3/CN1)
- **RX = GPIO35** (ingresso; GPIO35 è *input-only*, perfetto per un RX UART)

> Alternativa: liberare **GPIO16 (TX)** e **GPIO17 (RX)** rimuovendo il LED RGB (sono i pin UART2 "di default" dell'ESP32, comodi). Attenzione: su alcune schede **GPIO27 risulta non collegato** e il connettore **P1 (GPIO1/3)** è legato al CH340 USB, quindi poco pratico. GPIO21 è la retroilluminazione.

Nel codice si assegnano i pin con `CAT.begin(baud, SERIAL_8N2, RX_PIN, TX_PIN);`.

---

6. Architettura del software

Il firmware gira interamente sull'ESP32 del CYD e si compone di tre strati:

1. **Strato CAT** — funzioni che costruiscono i blocchi da 5 byte e li inviano sulla UART2 (`catSetFreq` , `catSetMode` , `catSetPTT` , `catReadStatus` , ...).
2. **Logica applicativa** — stato locale (frequenza, modo, banda), gestione dei pulsanti, polling periodico dello stato.
3. **Interfaccia utente** — TFT touch: display frequenza, tasti banda/modo/PTT, barra S-meter.

Librerie consigliate (Arduino IDE / PlatformIO):

- **TFT_eSPI** (Bodmer) per display + touch del CYD (configurare `User_Setup.h`).
- Nessuna libreria CAT: il protocollo è così semplice che conviene scriverlo a mano (5 byte per comando).

6.1 Configurazione minima di TFT_eSPI per il CYD

In `User_Setup.h` (o in `User_Setup_Select.h` selezionando un setup dedicato):

```
#define ILI9341_2_DRIVER      // molte CYD usano ILI9341; le più recenti ST7789
#define TFT_MISO 12
#define TFT_MOSI 13
```

```
#define TFT_SCLK 14
#define TFT_CS   15
#define TFT_DC    2
#define TFT_RST  -1
#define TFT_BL   21
#define TFT_BACKLIGHT_ON HIGH
#define SPI_FREQUENCY  40000000
// Touch (XPT2046)
#define TOUCH_CS 33
```

> Se il display resta nero o i colori sono invertiti, la tua CYD monta un **ST7789**: cambia il driver e, se serve, imposta `TFT_RGB_ORDER`.

6.2 Lo sketch di esempio

È allegato il file **`ft847_cyd_controller.ino`**: implementa lo strato CAT completo (frequenza, modo, PTT, lettura stato), una piccola GUI touch (frequenza, banda, modo, PTT, S-meter) e un **menu seriale** di test sulla USB, utile per collaudare la comunicazione prima di rifinire la grafica.

---

7. Messa in servizio passo-passo

1. **Imposta la radio.** Sul FT-847 vai nel menu **#37 [CAT RATE]** e scegli **4800 bps** (o 9600). Annota il valore.
2. **Collega l'hardware** come da schema (CYD → MAX3232 → DB-9 → FT-847), con **massa comune**.
3. **Alimenta** CYD (USB) e radio.
4. **Carica lo sketch** e apri il monitor seriale a **115200 bps** (USB).
5. **Test di comunicazione.** Dal menu seriale invia ad esempio `F 145500000` per portare la radio su 145,500 MHz. Se non succede nulla: * verifica che il baud coincida con il Menu #37; * **scambia TX↔RX**; * verifica la massa comune; * assicurati di usare il cavo **crossover**.
6. **Lettura stato.** Prova `R` per leggere frequenza/modo e `S` per l'S-meter (se la radio è una versione recente con capacità di lettura).
7. **Rifinisci la GUI** touch a piacere.

---

8. Limitazioni e cose da sapere

- **Solo controllo, non audio.** Il CAT comanda frequenza/modo/PTT e legge lo stato. Per i modi digitali (FT8, PSK…) serve **separatamente** un'interfaccia audio (jack DATA 3,5 mm per HF, mini-DIN 6 pin per packet FM) e il PTT può essere dato via CAT.
- **Niente split/VFO A-B/clarifier via CAT** sull'FT-847: il set comandi è limitato rispetto a radio più moderne (confermato da piRig/piWebCAT).
- **Versioni vecchie senza lettura:** se l'FT-847 è un esemplare "early" senza aggiornamento firmware, le letture di stato non rispondono; la scrittura sì.
- **Risoluzione 10 Hz:** la frequenza si imposta a passi di 10 Hz.
- **GPIO limitati:** sul CYD restano pochi pin; se il progetto cresce, valuta un ESP32 "liscio" + display SPI separato, oppure usa il **WiFi** del CYD per esporre una web-UI o per fare da ponte verso un PC (es. Hamlib via rete).
- **Isolamento:** per ambienti RF "sporchi" valuta un isolamento ottico/trasformatore sulla seriale, anche se l'FT-847 è già progettato per il collegamento diretto a un PC.

---

9. Varianti e idee

- **Controller da tavolo** con tasti fisici (encoder rotativo) oltre al touch.
- **Ponte WiFi → CAT:** il CYD fa da server web (o TCP) e inoltra i comandi CAT; così controlli la radio dal telefono o da un software di log sul PC.
- **Tracciamento satelliti:** l'FT-847 ha il modo satellite; un ESP32 può fare doppler-tuning via CAT (attenzione: il CAT non gestisce lo split, va usato il modo satellite con i VFO SAT RX/TX).
- **Logging:** memorizzare frequenza/modo/S-meter su SD del CYD.

---

10. Riferimenti

- Yaesu **FT-847 Operating Manual** — sezione *CAT System Programming* e *Rear Panel Connectors* (connettore CAT, formato

8N2, Menu #37, protocollo a 5 byte). Es.: http://www.radiomanual.info/schemi/YAESU_HF/FT-847_user.pdf

- **FT-847 CAT Command Chart** (opcode e parametri): https://f8khr.fr/images/_Partage/PstRotator/FT-847_Commandes_CAT.pdf
- **Hamlib** — backend `rigs/yaesu/ft847.c` (conferma BCD big-endian `freq/10`, tabella opcode, PTT, letture): https://github.com/Hamlib/Hamlib/blob/master/rigs/yaesu/ft847.c
- **piWebCAT** — note su FT-847/817/818 (comandi limitati, byte di risposta): https://www.piwebcat.g3vpx.net/52FT847FT817FT818.html
- **CYD pinout** (Random Nerd Tutorials): https://randomnerdtutorials.com/esp32-cheap-yellow-display-cyd-pinout-esp32-2432s028r
- **Digirig forum** — conferma cavo null-modem per CAT FT-847: https://forum.digirig.net/t/cables-for-yaesu-ft-847/257

Documento redatto per un progetto hobbistico di controllo CAT. Verifica sempre tensioni e piedinatura sul tuo esemplare di radio prima di collegare.